

Euclidean Distance

$$(x_i, y_i) \quad (x_j, y_j)$$

Manhattan Distance

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

$$|x_i - x_j| + |y_i - y_j|$$

01 Matrix

Given a $m \times n$ binary matrix **mat**, for each cell find the distance to the nearest cell with the value 0 such that distance b/w adjacent cells is 1.

$$x_i = 0 \\ y_i = 0$$

$$x_j = 2 \\ y_j = 2$$

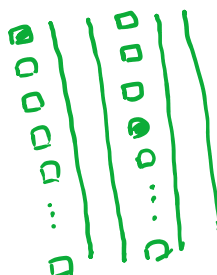
$$|0 - 2| + |0 - 2| = 4$$

	0	1	2	3	4
0	1	1	1	1	1
1	1	1	1	1	1
2	1	1	0	1	1
3	1	1	1	1	1
4	1	1	1	1	1

i/p

	0	1	2	3	4
0	4	3	2	3	4
1	3	2	1	2	3
2	2	1	0	1	2
3	3	2	1	2	3
4	4	3	2	3	4

o/p



	0	1	2	3	4
0	0	1	1	1	0
1	1	1	1	1	1
2	1	1	0	1	1
3	1	1	1	1	1
4	0	1	1	1	0

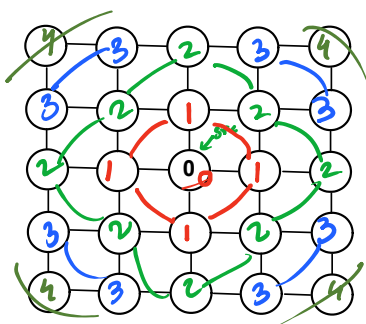
i/p

	0	1	2	3	4
0	0	1	2	1	0
1	1	2	1	2	1
2	2	1	0	1	2
3	1	2	1	2	1
4	0	1	2	1	0

o/p

	0	1	2	3	4
0	1	1	1	1	1
1	1	1	1	1	1
2	1	1	0	1	1
3	1	1	1	1	1
4	1	1	1	1	1

$m \times n$



time for single BFS call

$$O(V + 2E)$$

$$\#nodes \quad \#edges$$

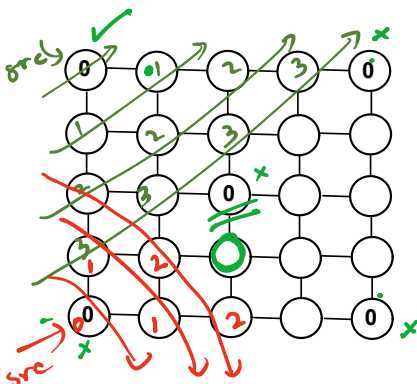
$$= mn + mn \sim O(mn)$$

In worst-case we'll
mn calls

$$\therefore \text{total time} = mn \cdot mn = O(m^2 n^2)$$

$$(m-1)n + m(n-1) \\ mn - n + mn - m \\ \sim 2mn \sim mn$$

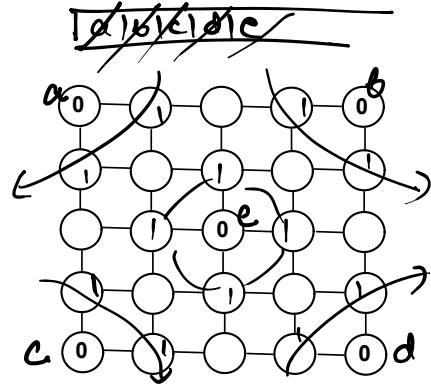
	0	1	2	3	4
0	0	1	1	1	0
1	1	1	1	1	1
2	1	1	0	1	1
3	1	1	1	1	1
4	0	1	1	1	0



Multi-Src BFS

src: → ↓ ↓ ↓

	0	1	2	3	4
0	0	1	1	1	0
1	1	1	1	1	1
2	1	1	0	1	1
3	1	1	1	1	1
4	0	1	1	1	0



[HW] Rotten Oranges

Rotten Oranges

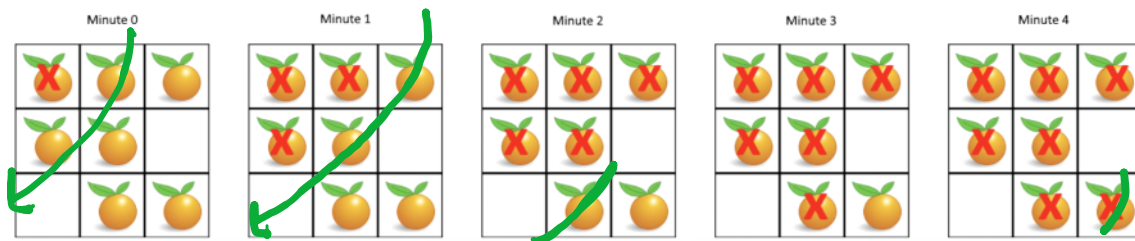
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1 .

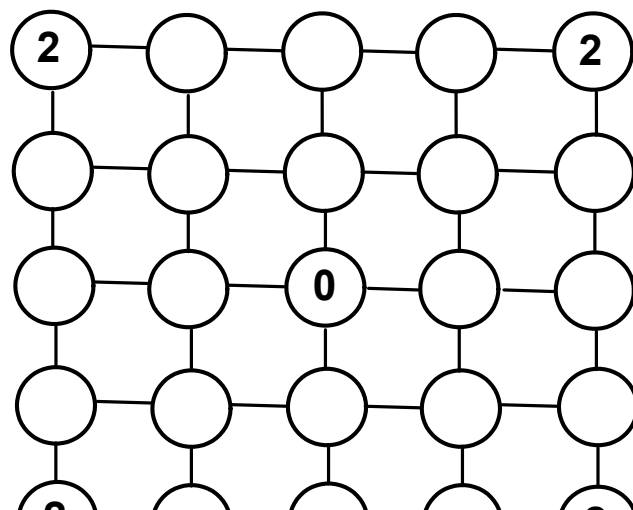
Example 1:



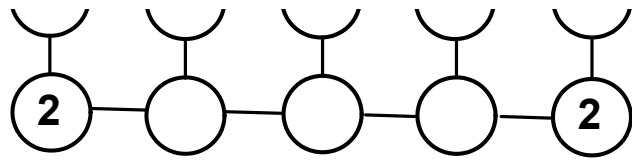
Input: grid = `[[2,1,1],[1,1,0],[0,1,1]]`

Output: 4

	0	1	2	3	4
0	2	1	1	1	2
1	1	1	1	1	1
2	1	1	0	1	1
3	1	1	1	1	1
4	2	1	1	1	2



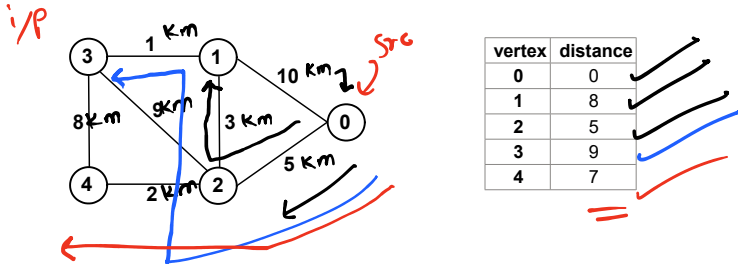
4	2	1	1	1	2
---	---	---	---	---	---



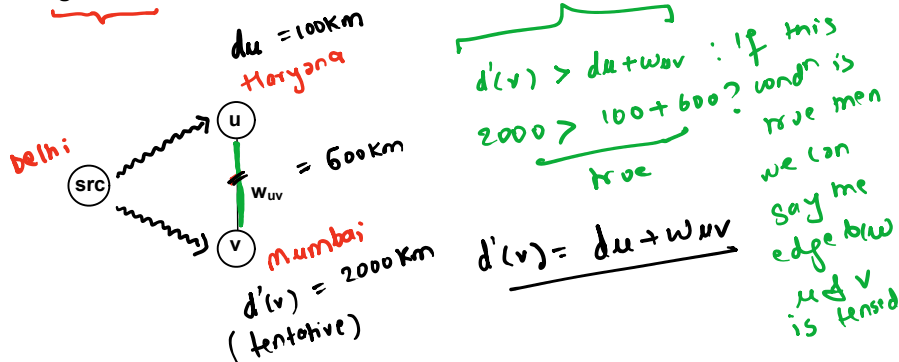
[HW] <https://codeforces.com/problemset/problem/20/C>
 [HW] <https://cses.fi/alon/task/1195>
 [HW] <https://leetcode.com/problems/number-of-ways-to-arrive-at-destination/>

SSSP using Dijkstra's Algorithm

Given a **weighted** graph and **source** vertex design an algorithm to find the length of the **shortest path** from the **source** vertex to all the **other** vertices.

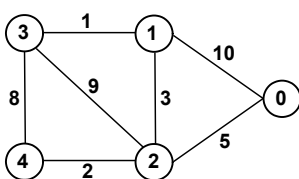


Tensed Edge and Edge Relaxation



Dijkstra's Algorithm

To solve the **SSSP** problem for **weighted graphs** using the **Dijkstra's** algorithm, iterate over all the vertices in the graph for which the **shortest distance** from the **source** has been **computed**, and **relax** all the **tensed edges** incident on them.

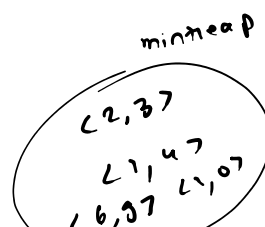


vertex	distance
0	0
1	∞
2	∞
3	∞
4	∞

Dijkstra's Implementation

To implement the **Dijkstra's** algorithm, we will maintain two data structures

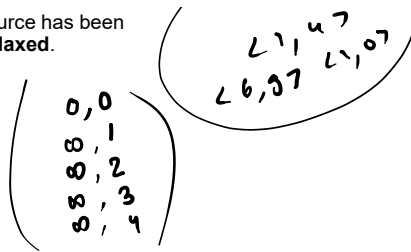
- To keep track of graph vertices that have been **explored**.
 - We will use a **Set** for this purpose.
 - A vertex is marked as explored if its **shortest distance** from source has been computed and all the **tensed edges** incident on it have been **relaxed**.
- To keep track of graph vertices that have not been explored.



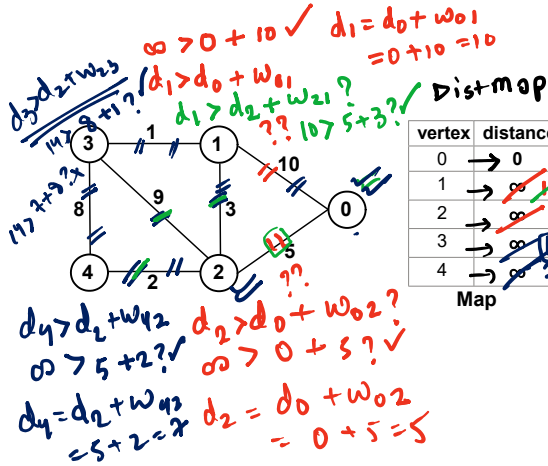
$(1, 0) \Rightarrow$ If first member is same then heap prop. is based on 2nd member of the pair.
 In a heap

- A vertex is marked as explored if its **shortest distance** from source has been computed and all the **tensed edges** incident on it have been **relaxed**.

- To keep track of graph vertices that have not been explored.
 - We will use a **Min-Heap** for this purpose.
 - Each item inside the **Min-Heap** will be a **pair** -
 - tentative shortest distance of vertex from source
 - label of vertex



In a heap of pairs, heap prop. is based on 1st element of pairs
on 2nd mem. of the pair



Dist-map

vertex	distance
0	0
1	10
2	5
3	11
4	7

Map

explored

0, 2
1, 3
4, 3

Set / Map

unexplored

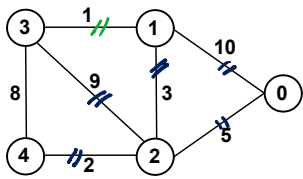
(0, 0)
(infinity, 1)
(infinity, 2)
(infinity, 3)
(infinity, 4)

min-Heap

(0, 0) dist node

(9, 3) dist node

(8, 1) dist node



Map

vertex	distance
0	0
1	10
2	5
3	9
4	7

explored

0, 2, 4

Set

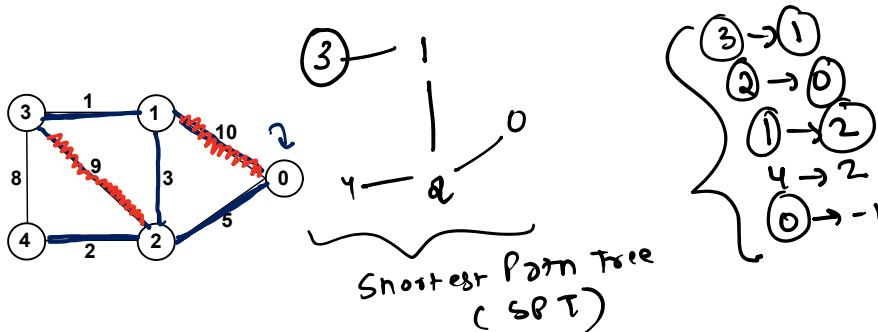
unexplored

(0, 0)
(infinity, 1)
(5, 2)

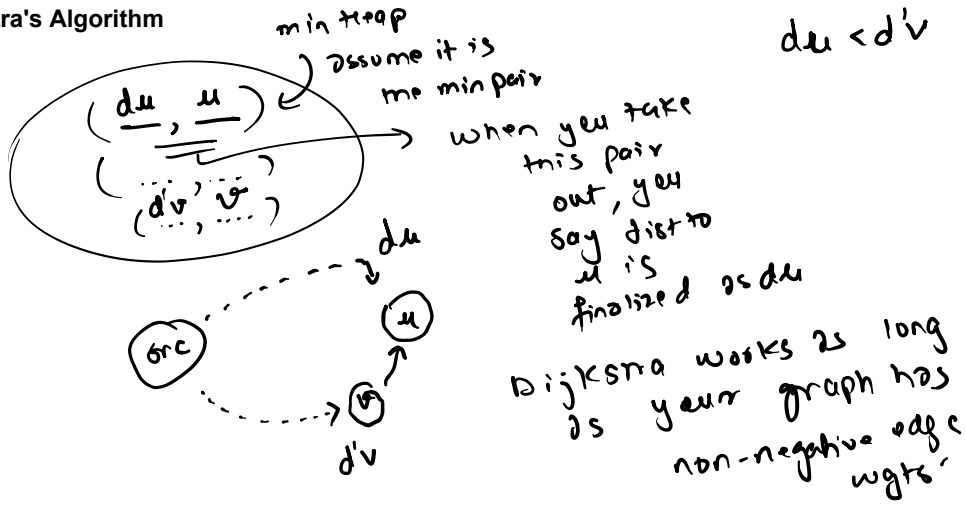
min-Heap

(0, 0) dist node

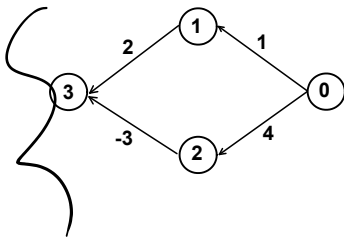
Path Construction



Correctness of Dijkstra's Algorithm



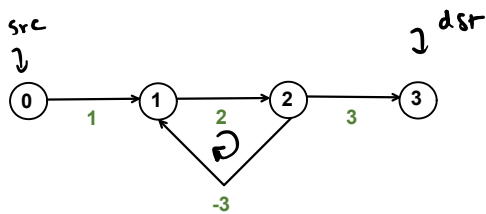
Dijkstra's algorithm doesn't work for graphs with a negative weight edge.



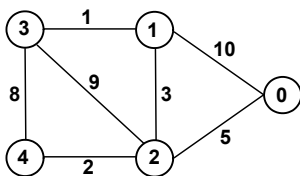
Is Dijkstra's algorithm exhaustive?

Greedy

For a **weighted** graph, the SSSP problem isn't defined if the graph contains a **negative weight cycle**.



[R][HW] Implementation of Dijkstra's algorithm using priority_queue<>



vertex	distance
0	0
1	∞
2	∞
3	∞
4	∞

Map

explored

Set

unexplored

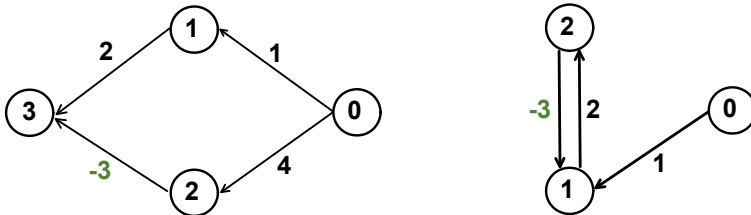
min-Heap



[HW] <https://leetcode.com/problems/cheapest-flights-within-k-stops/>

SSSP using Bellman-Ford Algorithm

The Bellman-Ford algorithm can solve the SSSP in case the graph contains **negative weight** edges. Also it can be used to detect a **negative weight cycle** in a graph.

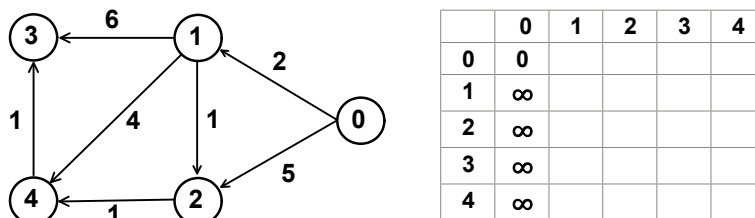


For a weighted graph, containing **V** vertices, the **minimum** no. of edges along the shortest path from the **source** to a **destination** vertex is **0** & the **maximum** number of edges along the shortest path is equal to **V - 1**.

If however, the number of edges along the **shortest** path between the **source** vertex and **destination** vertex exceeds **V - 1** then we can say the shortest path between the source and destination vertex contains a **negative weight cycle**.

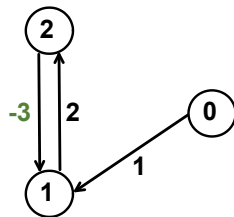
Bellman-Ford Algorithm

The idea behind the **Bellman-Ford** algorithm for solving the **SSSP** problem for **weighted** graph that contains **V** vertices is to **relax** all the its edges **V - 1** times.



How to detect a **negative** weight cycle using Bellman-Ford ?

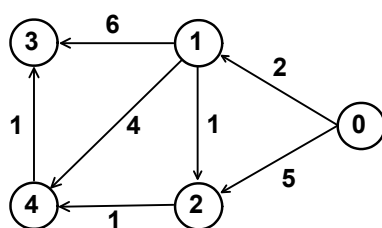
To **detect** a **negative weight cycle**, we **relax** all the edges of the graph once more. If the distances of ≥ 1 vertex from the source vertex **improve**, then we can conclude that the graph contains a negative weight cycle.



	0	1	2	3
0	0			
1	∞			
2	∞			

Can we use **Bellman-Ford** algorithm for **undirected** graph ?

Is Bellman-Ford algorithm exhaustive ?

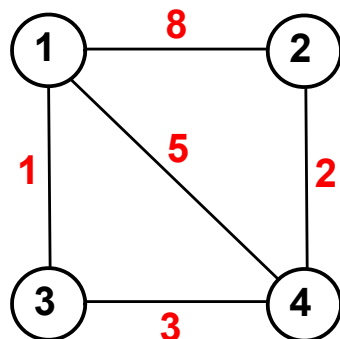


	0	1	2	3	4
0	0				
1	∞				
2	∞				
3	∞				
4	∞				

[HW] <https://codeforces.com/problemset/problem/295/B>

All Pairs Shortest Path using Floyd Warshall's Algorithm

Given a **weighted** graph, find the **shortest path** between all pairs of vertices.



	1	2	3	4
1	0	6	1	4
2	6	0	5	2
3	1	5	0	3
4	4	2	3	0

i

j

i

k

j

The Floyd-Warshall's algorithm can **detect** a **negative weight cycle** in a graph.

